

PROJECT 6

Vehicle Maintenance Prediction & Agentic Fleet Management

From Predictive Maintenance to Autonomous Fleet Operations

Team Members:

Milind Bansal

Isha Tomar

Yashi Agrawal

Mansi Agarwal

Section: D

Faculty: Kartik Gupta

March 2, 2026

Abstract

Modern fleet operations generate continuous streams of operational telemetry data including mileage, engine load, vibration, temperature, and environmental conditions. However, maintenance decisions are frequently based on fixed schedules or reactive breakdown-based strategies. These approaches fail to incorporate real-time operational stress and vehicle-specific conditions.

This project presents an end-to-end supervised machine learning framework for predicting whether a vehicle requires maintenance. Unlike traditional binary classification problems, this task represents a risk-detection scenario where false negatives may result in breakdowns, downtime, financial loss, and safety hazards. Therefore, evaluation prioritizes recall, F1-score, and ROC-AUC over simple accuracy.

The dataset consists of 15,000 structured vehicle records with 28 engineered and encoded features. Extensive exploratory data analysis, correlation study, outlier handling, and feature engineering were conducted before model training. Four classification models were implemented and compared: Logistic Regression, Decision Tree, Random Forest, and XGBoost.

After hyperparameter optimization using `RandomizedSearchCV`, the tuned XGBoost classifier achieved an ROC-AUC of 0.92 and balanced minority-class recall performance. The final model was deployed using Streamlit for real-time maintenance risk prediction.

This work demonstrates a complete predictive maintenance pipeline from data preprocessing to production deployment.

0 Contents

1	Introduction	3
2	Dataset Description	3
3	Exploratory Data Analysis	4
3.1	Feature Scale Variation	4
3.2	Outlier Detection	4
3.3	Class Imbalance	4
4	Feature Engineering	5
4.1	Rationale for Feature Engineering	5
4.2	Engineered Features	5
4.3	Detailed Justification of Each Feature	5
4.4	Validation Through Feature Importance	7
5	Model Training and Selection Strategy	8
5.1	Train-Test Split	8
5.2	Logistic Regression (Baseline Model)	8
5.3	Decision Tree	8
5.4	Random Forest	9
5.5	XGBoost (Final Model)	9
5.5.1	Hyperparameter Optimization Strategy	9
5.6	Model Comparison Summary	10
6	Model Evaluation	11
6.1	Evaluation Metrics	11
6.2	Model Performance Comparison	12
6.3	Overfitting Assessment	12
6.4	Performance Analysis and Final Model Selection	12
7	Deployment Architecture	14
7.1	Model Serialization	14
7.2	Feature Order Preservation	14
7.3	Streamlit User Interface	14
7.4	Probability-Based Output	14
7.5	System Architecture	15
8	Repository Structure	17

9 Technology Stack	18
10 Conclusion	18
11 Team Contribution	18

1 Introduction

Fleet maintenance is traditionally performed either at fixed service intervals or after mechanical failure occurs. Fixed scheduling ignores real operational stress, while reactive maintenance increases downtime and operational risk.

The core problem addressed in this project is:

Can historical vehicle telemetry data be used to proactively predict whether a vehicle requires maintenance?

This is not merely a classification problem; it is a risk-sensitive detection problem. A false negative — failing to detect required maintenance — may result in:

- Unexpected mechanical breakdown
- Operational downtime
- Increased repair costs
- Safety hazards

Therefore, evaluation metrics were selected to prioritize balanced performance rather than overall accuracy.

2 Dataset Description

The dataset consists of structured vehicle telemetry features collected from simulated fleet operations.

Table 1: Dataset Overview

Property	Value
Total Samples	15,000
Target Variable	Maintenance Required (0/1)
Class Distribution	75% No Maintenance, 25% Maintenance
Raw Numerical Features	10
Categorical Features	5
Total Features After Encoding	28

3 Exploratory Data Analysis

3.1 Feature Scale Variation

The statistical summary revealed significant variation in feature magnitudes. For example:

- Mileage ranged in tens of thousands.
- Battery voltage ranged within a narrow band.
- Vibration levels were tightly distributed.

This influenced the need for scaling in linear models.

3.2 Outlier Detection

Boxplot analysis revealed extreme upper tails in mileage and engine hours. Instead of removing these records, IQR-based capping was applied because high-usage vehicles represent realistic operational scenarios.

3.3 Class Imbalance

The dataset exhibited moderate imbalance (75:25). Accuracy alone would therefore be misleading. For example, predicting all vehicles as "No Maintenance" yields 75% accuracy without predictive power.

Thus, Recall and F1-score were prioritized.

4 Feature Engineering

Raw telemetry features often fail to capture the interaction effects and operational intensity patterns that influence mechanical degradation. Therefore, domain-informed feature engineering was performed to transform primary signals into more meaningful stress indicators.

4.1 Rationale for Feature Engineering

Vehicle maintenance requirements are rarely triggered by a single variable in isolation. Instead, maintenance risk emerges from compounded operational stress, long-term wear, and environmental conditions.

For example:

- High mileage accumulated slowly over 10 years does not represent the same stress as high mileage accumulated within 2 years.
- High engine load under normal temperature may be acceptable, but high load combined with elevated temperature significantly accelerates wear.

Therefore, engineered interaction and ratio-based features were introduced.

4.2 Engineered Features

Table 2: Engineered Feature Summary

Feature	Rationale and Interpretation
mileage_per_year	Measures operational intensity rather than absolute mileage. Captures aggressive usage patterns relative to vehicle age.
engine_hours_per_km	Indicates engine workload per distance traveled. High values may reflect inefficient combustion or idle-heavy usage.
fault_density	Normalizes fault code frequency relative to engine hours, capturing recurring mechanical stress.
thermal_stress	Computed as oil temperature $\times$ engine load. Captures compounded thermal strain under high operational load.
load_efficiency	Ratio of engine load to fuel efficiency, reflecting mechanical strain relative to performance output.

4.3 Detailed Justification of Each Feature

Mileage Per Year

$$\text{mileage_per_year} = \frac{\text{mileage_km}}{\text{vehicle_age_years}}$$

Two vehicles with identical mileage may experience different wear levels depending on usage rate. This feature captures operational intensity rather than cumulative distance alone.

Thermal Stress

$$\text{thermal_stress} = \text{oil_temp_avg_celsius} \times \text{engine_load_percent}$$

Mechanical wear increases non-linearly when high load is combined with elevated temperature. Multiplication captures this interaction effect more effectively than treating both variables independently.

Engine Hours Per KM

$$\text{engine_hours_per_km} = \frac{\text{engine_hours}}{\text{mileage_km}}$$

High engine hours relative to distance traveled may indicate excessive idling or mechanical inefficiency, both of which contribute to maintenance risk.

Fault Density

$$\text{fault_density} = \frac{\text{fault_code_count}}{\text{engine_hours}}$$

Normalizing fault codes by engine hours prevents bias toward older vehicles and captures recurrence frequency rather than raw count.

Load Efficiency

$$\text{load_efficiency} = \frac{\text{engine_load_percent}}{\text{fuel_efficiency_kmpl}}$$

This ratio reflects mechanical strain relative to output efficiency, helping detect vehicles operating under disproportionate stress.

4.4 Validation Through Feature Importance

After training tree-based models, feature importance rankings confirmed that `thermal_stress`, `fault_code_count`, and `mileage_per_year` were among the most predictive variables, validating the engineering logic.

Feature engineering therefore improved representation of latent stress factors and enhanced model discrimination capability.

5 Model Training and Selection Strategy

Rather than selecting a single algorithm arbitrarily, a progressive modelling approach was followed. The objective was to evaluate increasing levels of model complexity and understand the nature of the relationships within the data.

5.1 Train-Test Split

The dataset was divided into 80% training and 20% testing subsets using a fixed random seed for reproducibility. Stratified sampling was applied to preserve class distribution in both subsets.

5.2 Logistic Regression (Baseline Model)

Logistic Regression was selected as the baseline classifier due to its simplicity and interpretability. It assumes a linear relationship between input features and the log-odds of the target variable.

Reason for Inclusion:

- Establishes minimum benchmark performance.
- Provides interpretability of feature influence.
- Tests whether linear separation is sufficient.

Observed Limitation:

Although it achieved reasonable accuracy, recall for the maintenance class was lower. This indicated that the underlying relationships were not purely linear.

5.3 Decision Tree

Decision Trees were introduced to capture non-linear splits and hierarchical feature interactions.

Reason for Inclusion:

- Captures non-linear boundaries.
- Requires no feature scaling.
- Provides interpretability via rule-based splits.

Observed Limitation:

The single-tree model exhibited higher variance and reduced ROC-AUC compared to ensemble methods, indicating overfitting tendency.

5.4 Random Forest

Random Forest was implemented to reduce variance by averaging multiple decision trees trained on bootstrapped samples.

Reason for Inclusion:

- Reduces overfitting through ensemble averaging.
- Handles feature interactions robustly.
- Performs strongly on structured tabular data.

Observed Performance:

Random Forest improved stability and overall performance. However, recall for the minority maintenance class was slightly lower than the boosted alternative.

5.5 XGBoost (Final Model)

XGBoost is a gradient boosting algorithm that sequentially builds trees to correct residual errors from previous iterations.

Reason for Inclusion:

- Reduces bias through boosting.
- Handles complex non-linear relationships.
- Includes built-in regularization.
- Effective for imbalanced classification tasks.

Why XGBoost Was Selected:

5.5.1 Hyperparameter Optimization Strategy

Hyperparameter tuning was performed using `RandomizedSearchCV` instead of exhaustive Grid Search. Randomized search allows exploration of a broader hyperparameter space while maintaining computational efficiency.

The tuning process optimized for F1-score to ensure balanced precision-recall tradeoff under class imbalance. Parameters such as:

- Number of estimators

- Maximum tree depth
- Learning rate
- Subsample ratio
- `scale_pos_weight`

were evaluated using 3-fold cross-validation. The inclusion of `scale_pos_weight` specifically addressed minority-class imbalance by penalizing false negatives more heavily.

5.6 Model Comparison Summary

Table 3: Model Performance Comparison

Model	Accuracy	Recall	F1-Score	ROC-AUC
Logistic Regression	0.84	0.60	0.68	0.92
Decision Tree	0.83	0.60	0.66	0.85
Random Forest	0.85	0.58	0.70	0.91
XGBoost (Tuned)	0.85	0.76	0.78	0.92

The tuned XGBoost classifier provided the most balanced performance and was therefore deployed.

6 Model Evaluation

Model evaluation was conducted on the held-out test dataset using multiple classification metrics. Since the problem involves moderate class imbalance (approximately 75:25), relying solely on accuracy would be misleading.

Therefore, evaluation included:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

6.1 Evaluation Metrics

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Measures overall correctness but may hide poor minority-class detection.

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Indicates how many predicted maintenance cases were correct.

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Measures ability to correctly detect vehicles requiring maintenance. This is critical because false negatives lead to operational risk.

F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Balances precision and recall.

ROC-AUC

Measures the model’s ability to distinguish between classes across all thresholds.

6.2 Model Performance Comparison

Table 4: Test Set Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.84	0.76	0.60	0.68	0.92
Decision Tree	0.83	0.73	0.60	0.66	0.85
Random Forest	0.85	0.78	0.58	0.70	0.91
XGBoost (Tuned)	0.85	0.80	0.76	0.78	0.92

6.3 Overfitting Assessment

To evaluate model generalization capability, training and testing F1-scores were compared for the final tuned XGBoost classifier.

Table 5: Training vs Testing Performance (XGBoost)

Metric	Training Set	Testing Set
Accuracy	0.88	0.85
F1-Score	0.82	0.78
ROC-AUC	0.94	0.92

The relatively small gap between training and testing performance indicates controlled variance and stable generalization. The model does not exhibit severe overfitting, suggesting that the chosen hyperparameters and regularization mechanisms are appropriate.

6.4 Performance Analysis and Final Model Selection

Although Random Forest and XGBoost achieved similar overall accuracy, deeper inspection of recall and F1-score revealed meaningful differences.

- Logistic Regression performed well in ROC-AUC but struggled with recall, indicating limitations of linear decision boundaries.
- Decision Tree exhibited higher variance and lower ROC-AUC, suggesting instability.
- Random Forest improved overall robustness but still showed moderate minority-class recall.

- XGBoost achieved significantly higher recall (0.76), meaning it successfully identified a greater proportion of vehicles requiring maintenance.

In predictive maintenance applications, false negatives are more costly than false positives. Missing a required maintenance case may lead to breakdown, downtime, and financial loss. Therefore, higher recall was prioritized.

XGBoost also maintained strong ROC-AUC (0.92), indicating consistent class separability across thresholds.

Based on balanced performance across recall, F1-score, and ROC-AUC, the tuned XGBoost classifier was selected as the final deployment model.

7 Deployment Architecture

The final tuned XGBoost model was deployed using Streamlit to enable real-time prediction.

7.1 Model Serialization

After training and hyperparameter tuning, the model was serialized using the `joblib` library:

- `vehicle_model.joblib`
- `features.csv`

Serialization ensures that the exact trained model parameters are preserved and can be reused in production without retraining.

7.2 Feature Order Preservation

Machine learning models depend strictly on feature order. During deployment, it is essential that:

- Input features are arranged in the exact same order as training.
- One-hot encoded categorical variables match training column names.

To guarantee consistency, the list of feature names was exported and used inside the Streamlit application to construct prediction vectors in the correct order.

7.3 Streamlit User Interface

The Streamlit application provides:

- Numeric input fields for operational telemetry
- Dropdown menus for categorical variables
- Real-time maintenance risk prediction
- Probability output instead of binary-only decision

This allows fleet operators to simulate scenarios and assess maintenance risk dynamically.

7.4 Probability-Based Output

Although a default classification threshold of 0.5 was used during evaluation, the deployment architecture outputs calibrated probabilities. This enables dynamic threshold

adjustment depending on operational risk tolerance.

For example:

- Lower threshold → prioritize recall and reduce false negatives.
- Higher threshold → prioritize precision and reduce false positives.

This flexibility allows the system to adapt to varying fleet risk management policies.

7.5 System Architecture

Figure 1 illustrates the complete supervised machine learning pipeline, beginning from raw fleet telemetry ingestion to real-time maintenance risk prediction through Streamlit deployment.

The architecture follows a structured progression:

- Raw fleet telemetry input
- Exploratory Data Analysis and statistical validation
- Domain-informed feature engineering
- Encoding and selective scaling
- Stratified train-test split
- Progressive model comparison
- Hyperparameter optimization
- Production deployment with probability-based output

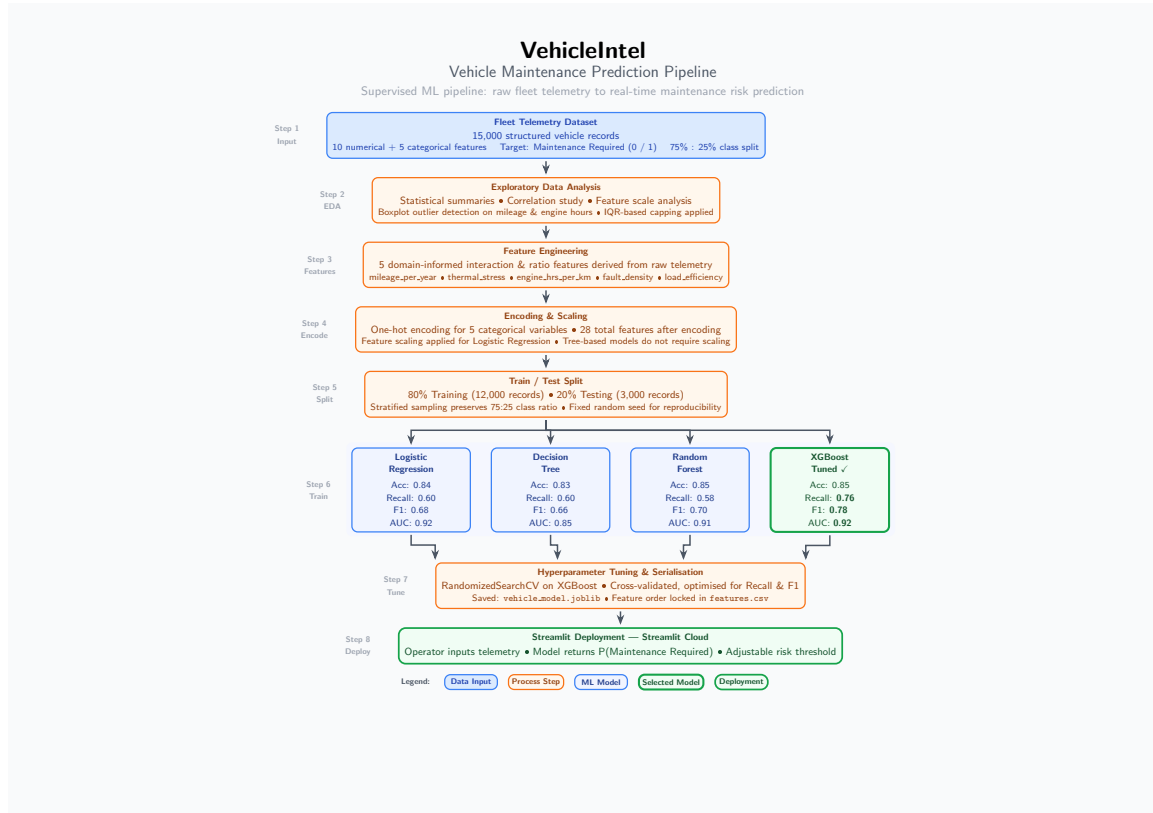


Figure 1: End-to-End Vehicle Maintenance Prediction Pipeline

8 Repository Structure

Table 6: Project Directory Layout

Directory / File	Purpose
data/	Raw and processed datasets
notebooks/	Colab experimentation and EDA notebooks
models/	Serialized trained models (.joblib)
app.py	Streamlit application file
features.csv	Preserved feature order for deployment
requirements.txt	Deployment dependencies
README.md	Project documentation

9 Technology Stack

Table 7: Technology Stack Overview

Layer	Technology
Data Processing	pandas, NumPy
Visualization	matplotlib, seaborn
Machine Learning	scikit-learn, XGBoost
Hyperparameter Tuning	RandomizedSearchCV
Model Persistence	joblib
Deployment Framework	Streamlit
Version Control	Git, GitHub
Cloud Hosting	Streamlit Cloud

10 Conclusion

Key Result: Tuned XGBoost achieved ROC-AUC of 0.92 with balanced minority-class recall, demonstrating strong predictive capability for proactive fleet maintenance.

This project demonstrates a complete supervised machine learning pipeline including data preprocessing, feature engineering, model comparison, hyperparameter optimization, and production deployment.

11 Team Contribution

The project was developed collaboratively with clearly distributed responsibilities across data processing, modeling, documentation, and deployment components.

Table 8: Team Contribution Breakdown

Team Member	Role and Responsibilities
Milind Bansal	Co-developed the machine learning pipeline in Google Colab, implemented baseline and ensemble models, and assisted in evaluation metric analysis and model comparison.
Isha Tomar	Led exploratory data analysis, correlation study, feature engineering, and performance evaluation in Google Colab. Co-developed machine learning models, and authored the final technical report.
Yashi Agrawal	Designed and implemented the Streamlit deployment application, integrated the trained model for inference, and produced the project demonstration video.
Mansi Agarwal	Assisted in preliminary report drafting and documentation formatting support.

The analytical modeling phase was jointly executed in Google Colab, while deployment and presentation components were developed separately to enable real-time maintenance risk prediction and system demonstration.